

Truck APS failure prediction

Sean Moriarty

CSCI 460

5/4/26

Abstract:

This project addresses the problem of predicting whether a truck failure is caused by the Air Pressure System (APS), for this project the sensor data from the APS Failure at Scania Trucks dataset was used to train machine learning models. This dataset is highly imbalanced, with only a small proportion of the data representing failures, this made accurate detection of the minority class important. The primary objective of this study is to minimize false negatives, as missed failures can have significant operational consequences.

Exploratory Data Analysis (EDA) was conducted to examine class imbalance, missing data, and feature relationships. Preprocessing steps included removal of features with excessive missing values, median imputation, and feature scaling. Principal Component Analysis (PCA) was evaluated for dimensionality reduction but was ultimately not included in the final model due to limited performance gains. Multiple supervised learning models were evaluated using cross-validation, including Logistic Regression, Support Vector Machines, K-Nearest Neighbors, Decision Trees, Random Forests, and SGD Classifier.

Recall was the metric primarily focused on for model evaluation, Logistic Regression and SGD Classifier were selected for hyperparameter tuning. Logistic Regression was chosen as the final model due to its strong balance between recall and precision. On the held-out test dataset, provided with training data, the model achieved approximately 93% recall, successfully identifying the majority of APS failures while maintaining reasonable precision. These results demonstrate that Logistic Regression provides an effective and reliable solution for minimizing missed failures (false negatives).

1. Introduction:

1.1 Problem Definition:

This project is addressing a supervised learning binary classification problem. It's attempting to predict whether or not, given the data, a truck's failure was caused by the Air Pressure System (APS). In particular, according to the description included with the data, missed failures (false negatives) are the most costly and minimizing them is the primary goal. Making recall the primary evaluation metric.

1.2 Dataset Description:

The dataset is the APS Failure dataset from Scania Trucks, originally part of the 2016 Industrial Data Analysis (IDA) Challenge. The data was collected from trucks operating in real world conditions and under everyday usage and with a focus on their APS systems which are used for braking and gear shifting.

The raw dataset contains 60,000 training instances and 171 features. Each instance represents a truck's state and sensor readings.

The target variable, labeled *class*, indicates whether a failure occurred in the APS system. The positive class ("pos") represents failures related specifically to APS components, while the negative class ("neg") represents failures unrelated to the APS system or no failure.

1.3 Objective:

The objective of this project is to develop and evaluate machine learning models capable of predicting likely APS system failures in trucks based on sensor data. A focus is to minimize misclassifications, particularly false negatives, where a likely failure is not detected.

According to the dataset, missing a failure (false negative) is significantly more costly than incorrectly predicting a failure (false positive). Therefore, the project will compare multiple classification algorithms to determine which model provides the best prevention of false negatives. To accomplish model training will prioritize recall while maintaining a balance with precision that should be reflected in the F1-score.

1.4 Report Structure:

The rest of this report is structured as follows: The next section presents the Exploratory Data Analysis, including an examination of class imbalance, missing data, and feature relationships. This is followed by a methodology section, which details the preprocessing steps, feature transformations, model selection, and hyperparameter tuning process. The results and discussion section then presents the performance of the evaluated models and analyzes their effectiveness in minimizing false negatives. Finally, the report concludes with a summary of key findings and suggestions for future work.

2. Exploratory Data Analysis

2.1 Class Distribution

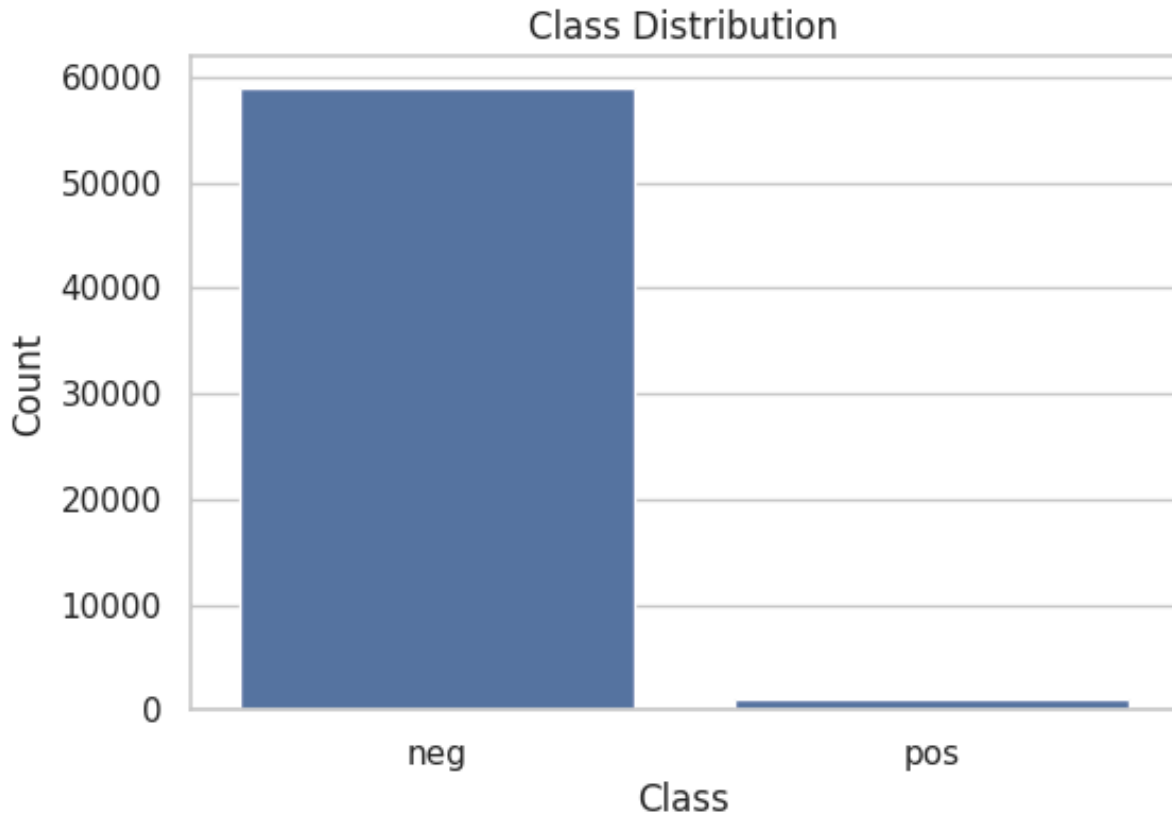


Fig. 1: target feature imbalance

The distribution of the target variable in the graph in Figure 1 shows a very high class imbalance. Out of 60,000 total instances, 59,000 belong to the negative class (“neg”) representing no failure or failure not related to the APS and only 1,000 belong to the positive class (“pos”) representing failure caused by the APS failing.

The imbalance in the data indicates that APS failures are rare events, accounting for about 1.67% of the dataset. It follows that standard accuracy is not a reliable evaluation metric for this data, because a model could achieve high accuracy by just mainly predicting the majority class. Therefore, metrics like recall are more appropriate, especially given the importance of detecting APS failures.

2.2 Missing Values Analysis

The dataset contains a substantial amount of missing values, with a total of 850,015 missing across all features. Given the large amount of data, 171 features (columns) and 60,000 entries (rows) there are 10,260,000 values. This means that about 8.29% of the data is missing ($850,015 / 10,260,000 \approx 0.0829$) therefore missing values are an issue with the data.

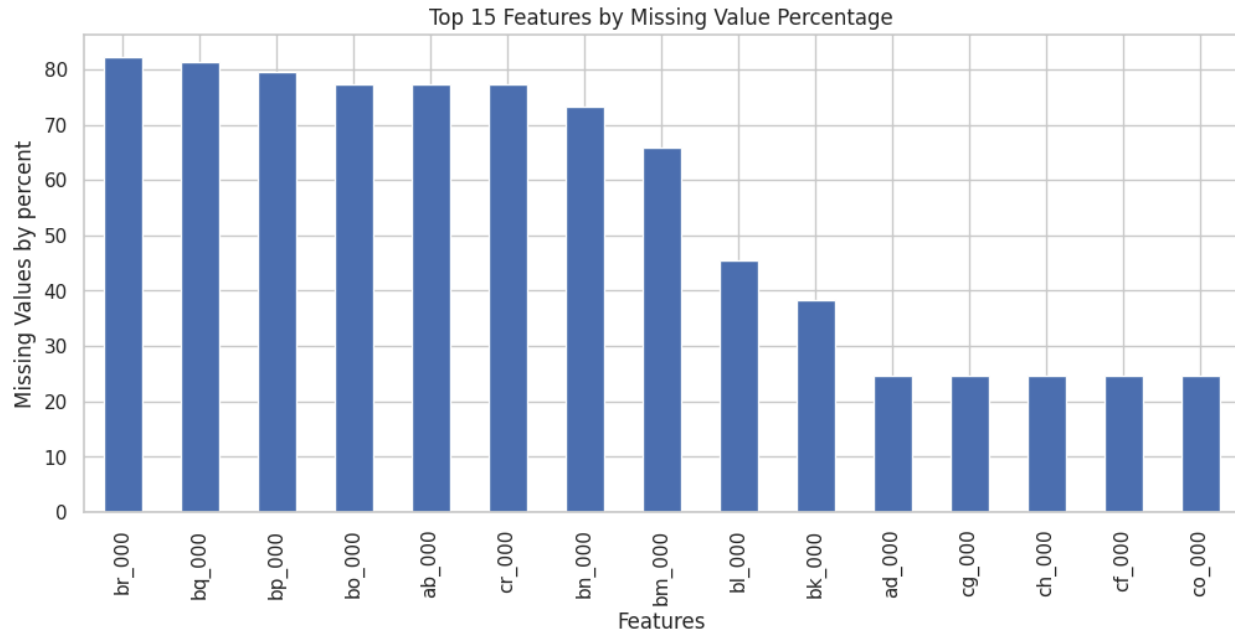


Fig 2: Missing values

Further analysis of data revealed that several features contained a very high percentage of missing data. This can be seen in Figure 2 which shows the top 15 features with the most missing data, two of which exceed 80%. To compensate for this, features with more than 70% missing values were removed. Because, imputing such a large portion of the missing data in these features would likely introduce noise and reduce the models' reliability.

Features with levels of missing data below 70% were retained and imputed using the median, because this approach handles outliers well and preserves the distribution of the data. This strategy helped in minimizing the impact of missing data on model performance.

2.3 Feature Correlation

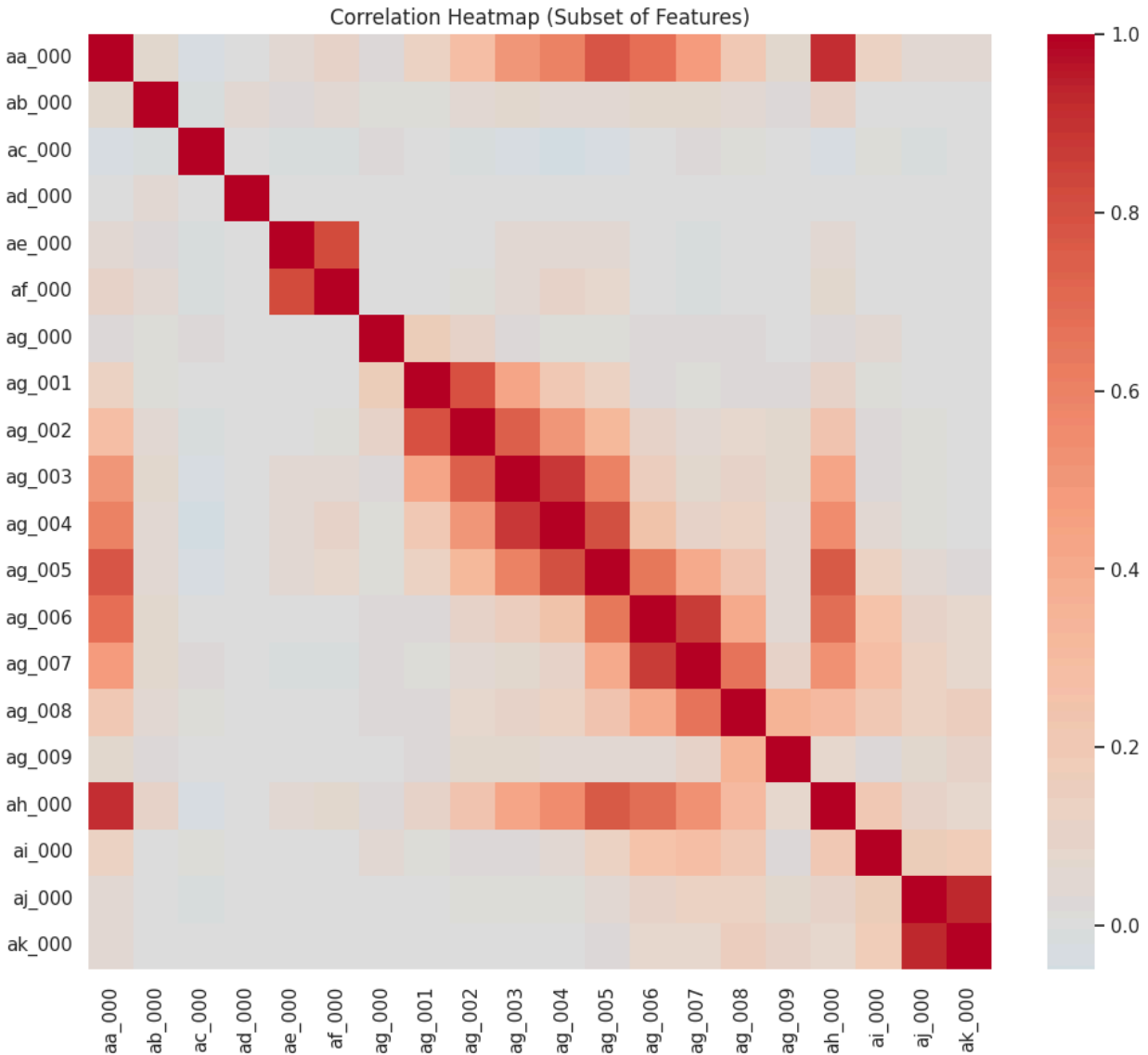


Fig. 3: Correlation heatmap

The correlation heatmap in Figure 3 reveals that while most features exhibit low correlation, a subset of features shown (ag) are highly correlated with one another. Meaning, there is redundancy within the features shown here and therefore possibly there is redundancy in other features in the data. This means that dimensionality reduction techniques such as Principal Component Analysis (PCA) or feature selection may be beneficial for improving model efficiency and reducing redundancy

2.4 Insights

Exploratory data analysis revealed several characteristics of the dataset that directly influenced my modeling decisions. First, the extreme class imbalance (only ~1.67% positive cases) made it clear to me that a standard evaluation metric, like accuracy, would not be appropriate.

Second, the presence of missing data across many features required thoughtful handling. While the overall missing percentage was moderate (~8%), certain features had very high levels of missing data. This uneven distribution made feature removal and imputation a necessary preprocessing step.

Third, the correlation analysis showed me that while most features were weakly correlated, some groups of features exhibited strong relationships. This suggested that there was redundancy in the data, this motivated my exploration of dimensionality reduction techniques such as PCA. However, later results showed that this redundancy did not meaningfully impact model performance.

Overall, the EDA phase highlighted to me the need for careful preprocessing, appropriate metric selection, and model choices that could handle high-dimension and imbalanced data.

3. Methodology

3.1 Data Preprocessing:

To clean and prepare the data for modeling I took several steps. First, for missing values I dropped the features with over 70% missing data. This was to avoid introducing too much noise when later imputing to populate the remaining missing data and possibly reducing the models reliability.

Next, I imputed the remaining features using the median. Median imputation was used because it isn't affected much by outliers and maintains the data's distribution. Which, given the large range of sensor data values, seemed appropriate.

After imputation, I scaled the training data using standardization. This step was necessary because many of the machine learning models used, such as Logistic Regression, SVM, and SGD Classifier, are sensitive to the scale of the data's input features. Standardization helped ensure that all features contributed to the models training.

Finally, Principal Component Analysis was also evaluated to potentially reduce the dimensionality and redundancy of the data. While PCA did reduce the number of features, it did not meaningfully improve the models' performance. Therefore, PCA was not included in the final model.

3.2 Feature Engineering & Selection

I did explore the use of PCA to reduce the dimensionality and redundancy of the data. Both 99% and 95% data preservation were explored and I paid particular attention to the behavior of the most relevant metrics, recall and precision. Recall was the main metric I paid attention to, because of the importance of minimizing false negatives.

While PCA did reduce dimensionality, the impact on model performance was small, with minimal or no improvement seen in several models. This suggested that the original features already captured the relevant information effectively. So, I didn't use PCA in the training of the final models, only in early selection

3.3 Machine Learning Models

I implemented and evaluated six supervised learning models to address this binary classification problem, including Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, Random Forests, and SGDClassifier. Model performance was assessed using cross-validation with metrics including accuracy, precision, recall, and F1-score, with focus on recall due to the importance of minimizing false negatives to the problem.

Logistic Regression is a linear model that estimates the probability of class membership using a logistic function. It is suited for use in binary classification problems and provides a strong baseline because of its simplicity and interpretability.

K-Nearest Neighbors (KNN) is a distance based algorithm that classifies instances based on the majority class of the instances nearest neighbors. It is useful for capturing local patterns in the data but can be sensitive to feature scaling and high dimensionality. Issues that likely explain its poor performance in this project.

Support Vector Machines (SVM) aim to find an optimal hyperplane that maximally separates the two classes. SVMs are effective in high dimensional spaces and can model complex boundaries.

Decision Trees are non-linear models that recursively split the data based on feature values to create a tree-like structure. They are easy to interpret and can capture complex relationships in the data.

Random Forests are an ensemble method that builds multiple decision trees and combines their predictions to improve generalization and reduce overfitting

An SGD Classifier is a linear model trained using stochastic gradient descent. It is computationally efficient for large datasets.

These models were selected because they are all supervised learning algorithms appropriate for a binary classification problem and because they represent a diverse set of approaches, including linear, distance-based, and ensemble methods, which enabled a more comprehensive evaluation of the classification problem.

3.4 Experimental Setup

The dataset was divided into training and testing subsets using an 80/20 split and a random state of 42. Stratification was applied to preserve the original class distribution. This ensured that both the training and test sets reflected the underlying class imbalance present in the data. Additionally, class weight was set to “balanced” to help account for the imbalance in training sets target feature. The training set was used for model development, including preprocessing, model selection, and hyperparameter tuning. While the test set was used for testing models after tuning.

To further assess model performance and reduce the risk of overfitting, k-fold cross-validation was applied to the training data, using 5 folds. This approach allowed each model to be evaluated across multiple splits of the data, providing a better estimate of performance.

In addition to the internal train/test split, a separate test dataset provided with the problem was used for final evaluation. This dataset was not used during training or model selection, ensuring an unbiased assessment of the model’s ability to generalize to unseen data.

All experiments were conducted using Python. Key libraries included pandas and NumPy for data manipulation, scikit-learn for preprocessing, model implementation, and evaluation, and Matplotlib and Seaborn for data visualization.

3.5 Hyperparameter Tuning

After the initial cross-validation results, only the Logistic Regression model and the SGD Classifier model were selected for tuning due to their strong recall performance. The other models did not show enough promise.

To tune these models I used a grid search combined with 5-fold cross-validation. This allowed me to evaluate different hyperparameter combinations. I focused on Recall as the primary metric during tuning to ensure I was minimizing false negatives.

For Logistic Regression, the regularization strength parameter (C) was tuned over a range of values, along with different solver options. This allowed the model to balance underfitting and overfitting while maintaining stability during training.

For the SGDClassifier, key hyperparameters such as the loss function, regularization strength (alpha), and learning rate were experimented with. These parameters influence how the model converges and generalizes.

I selected the best-performing hyperparameter configurations based on the cross-validated recall scores. These models were then evaluated on the test set and the provided external test dataset to assess their generalization performance.

3.6 Evaluation Metrics:

I analyzed several evaluation metrics to assess model performance such as accuracy, precision, recall, and F1-score. These metrics provided complementary information on classification performance, particularly in the context of class imbalance.

Recall was the primary evaluation metric, recall is the proportion of actual positive cases correctly identified by the model. In this problem, recall was critical because it directly measured the model's ability to detect APS failures. Also, since false negatives (missed failures) are the most costly type of error, maximizing recall was the primary objective.

Precision measures the proportion of predicted positive cases that are actually positive. While precision is less critical than recall in this problem, it is still important for understanding the rate of false positives. For example, a low precision would indicate that the model produces many false alarms, which might have annoying implications in the real-world.

The F1-score, which is the harmonic mean of precision and recall, was used as a secondary metric to evaluate the balance between these two measures. This was particularly useful when comparing models that achieve similar recall but differed in precision.

Accuracy was also reported for completeness; but, it was not considered a reliable metric due to the severe class imbalance in the dataset. Because a model could achieve high accuracy by predominantly predicting the majority class while failing to detect APS failures.

In addition to these metrics, I created a confusion matrix to provide a detailed breakdown of model predictions, including true positives, true negatives, false positives, and false negatives. This allowed me to directly interpret the types of errors made by the model and ensured alignment with the project's objective of minimizing false negatives.

4. Results and Discussion:

4.1 Model Performance Comparison

I evaluated the performance of the tuned models, Logistic Regression and SGDClassifier, using both the internal test split and the provided external test dataset. Table 1 summarizes the results across accuracy, precision, recall, and F1-score.

Table 1: Performance of Tuned Models

	Model	Dataset	Accuracy	Precision	Recall	F1	
high strong	Logistic Regression	Internal Test	0.97	0.36	0.94	0.52	Both models achieved recall, implying
	SGDClassifier	Internal Test	0.96	0.27	0.96	0.42	
	Logistic Regression	Final Test	0.98	0.49	0.93	0.64	
	SGDClassifier	Final Test	0.96	0.35	0.95	0.51	

performance in detecting APS failures. While the SGD Classifier model achieved slightly higher recall, the Logistic Regression model demonstrated meaningfully higher precision and F1-score, this indicated a better overall balance between correctly identifying failures and limiting false positives.

4.2 Visualization of Model Performance

Figure 4 illustrates the comparison of recall across models and feature sets.

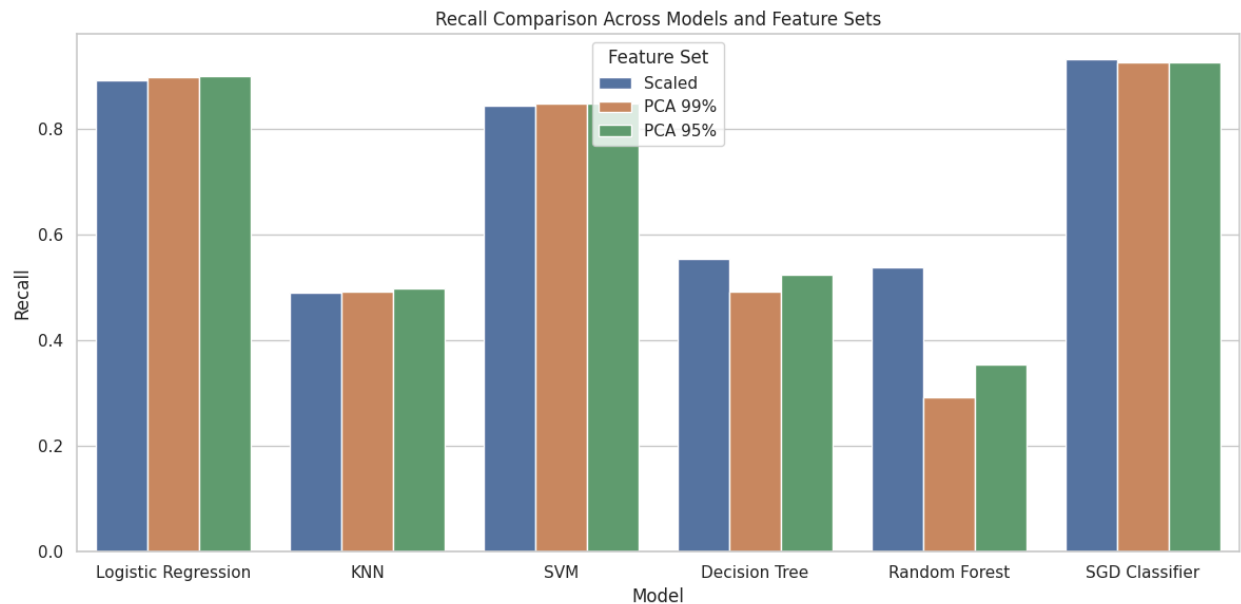


Fig. 4: Recall

Figure 4 provides a Recall comparison across models and feature sets. Logistic Regression and SGD Classifier consistently achieved the highest recall values, with minimal impact observed from PCA-based dimensionality reduction. The performance of Logistic Regression and SGD here is the primary reason they were selected for hyperparameter tuning.

Figure 5 illustrates the comparison of Precision across models and feature sets.

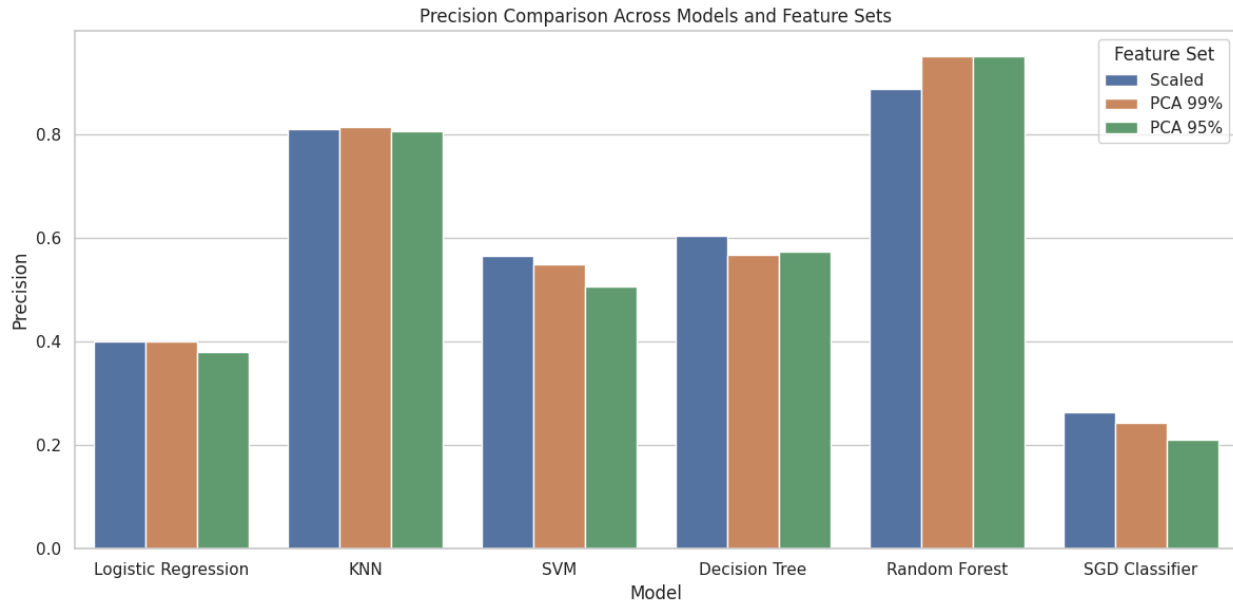


Fig. 5: Precision

Figure 5 provides comparison across models and feature sets of model precision. K-Nearest Neighbors and Random Forest consistently achieved the highest precision values.

Figure 6 illustrates the comparison of F1-score across models and feature sets.

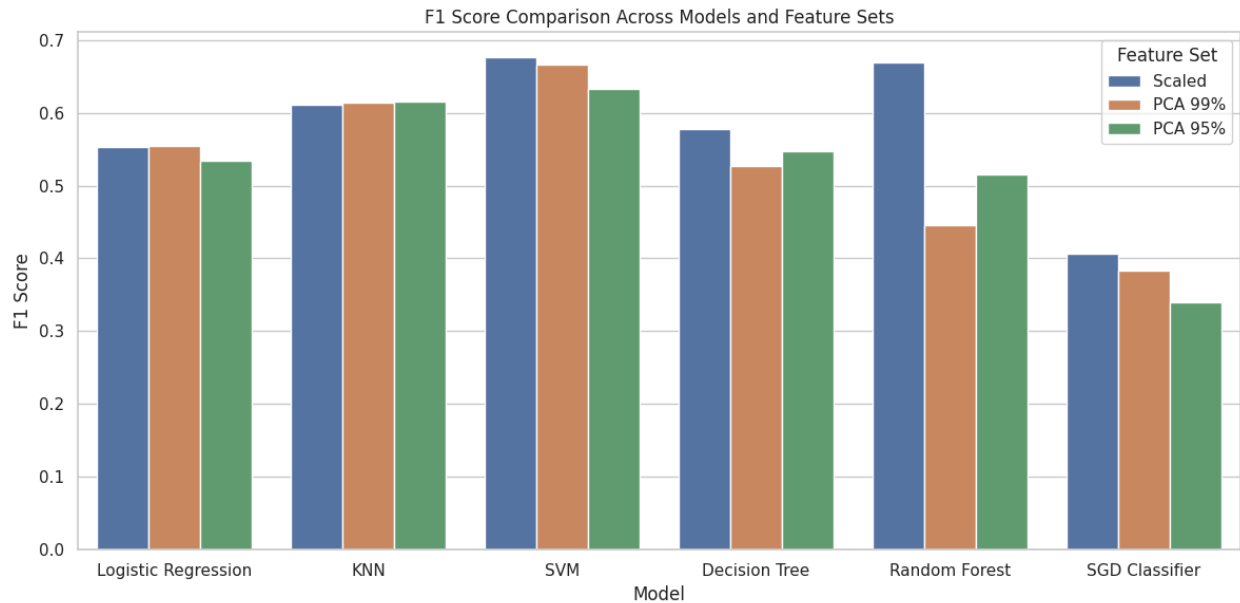


Fig. 6: F1-score

Figure 6 provides comparison across models and feature sets of F1 scores. SVM and K-Nearest Neighbors achieved the highest F1 scores values.

Figure 7 illustrates the comparison of F1-score across models and feature sets.

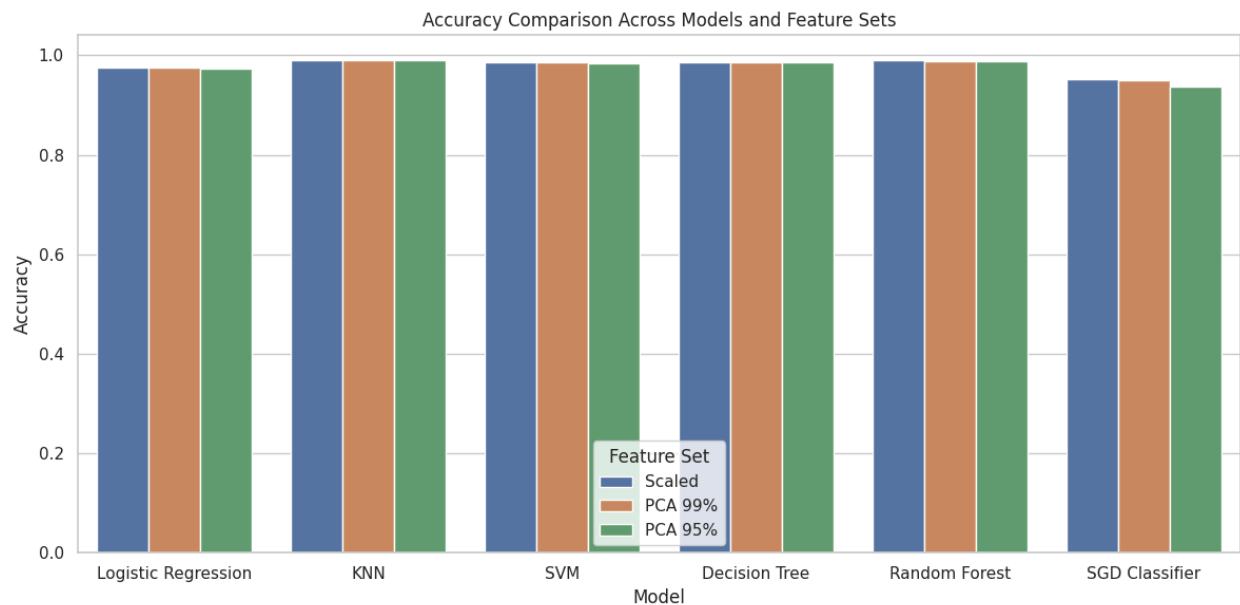


Fig. 7: accuracy

Figure 7 provides comparison across models and feature sets of the accuracy metric. This graph demonstrates why accuracy is not a good metric for this problem.

4.3 Confusion Matrix Analysis

To better understand model performance, I analyzed confusion matrices for the final models on the internal test dataset and then the hold-out test data set provided.

Figure 8 illustrates the Best SGD on the internal test set.

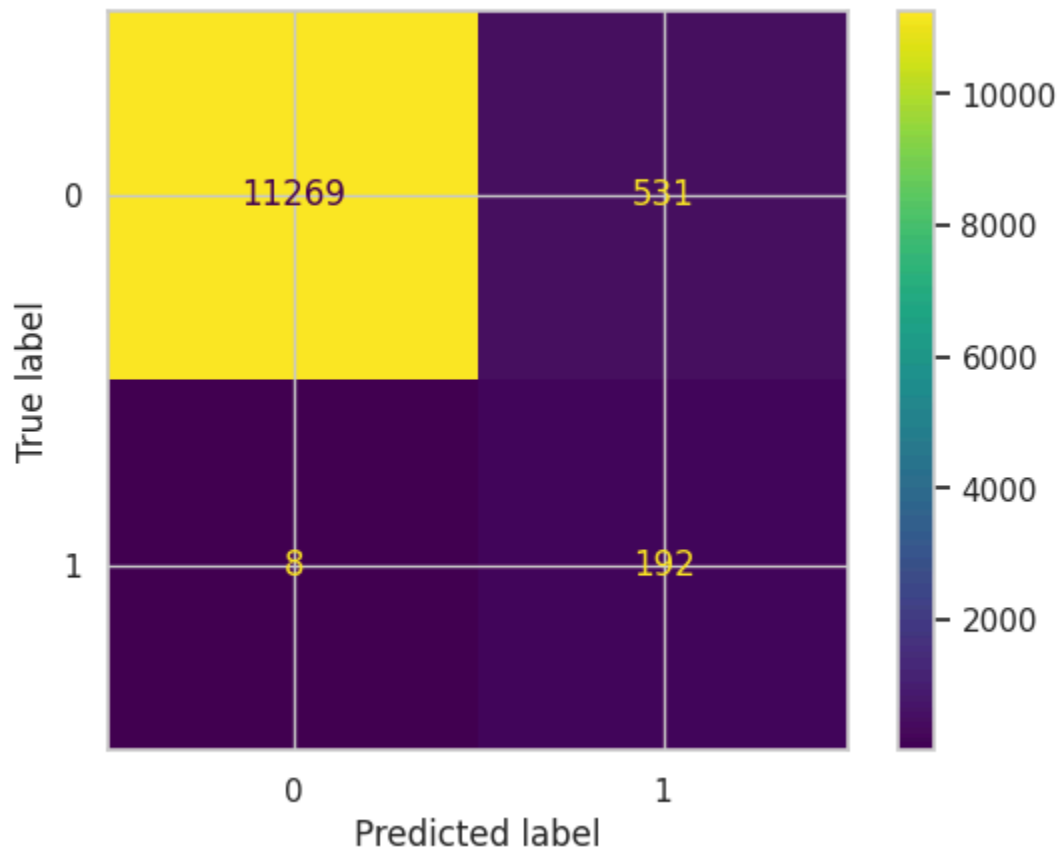


Fig. 8: best SGD - internal

The confusion matrix for the SGD Classifier in Figure 8 was trained on the internal test set. It shows that the model correctly classified 192 APS failures while missing only 8 cases (false negatives). This corresponds to a high recall, which is consistent with the SGD Classifier being effective at detecting failures. However, the model produced 531 false positives, meaning that a decent number of non-failure cases were incorrectly classified as failures by the model.

Figure 9 illustrates the best Logistic Regression model on the internal test set.

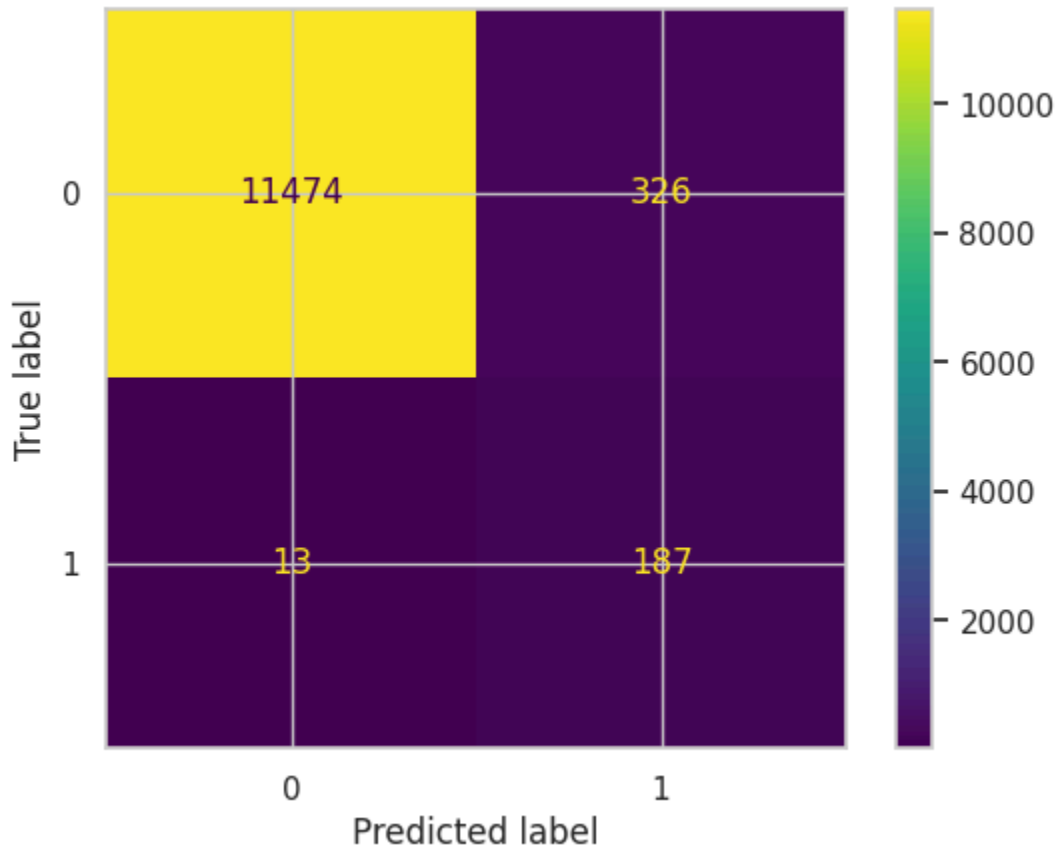


Fig. 9: best Logistic Regression - internal

In comparison, the Logistic Regression model trained on the internal test set demonstrates more balanced behavior. As shown in Figure 9, the model correctly identified 187 APS failures while missing 13 cases (false negatives). While this is slightly worse than the SGD Classifier in terms of recall, the difference is small.

There is a meaningful difference in the number of false positives classified by the model. Logistic Regression produced 326 false positives, which is significantly lower than the 531 observed for SGDClassifier.

Overall, the confusion matrix suggests to me that the Logistic Regression model provides a better tradeoff between detecting failures and limiting false alarms.

Figure 10 illustrates the best SGD model on the real test set.

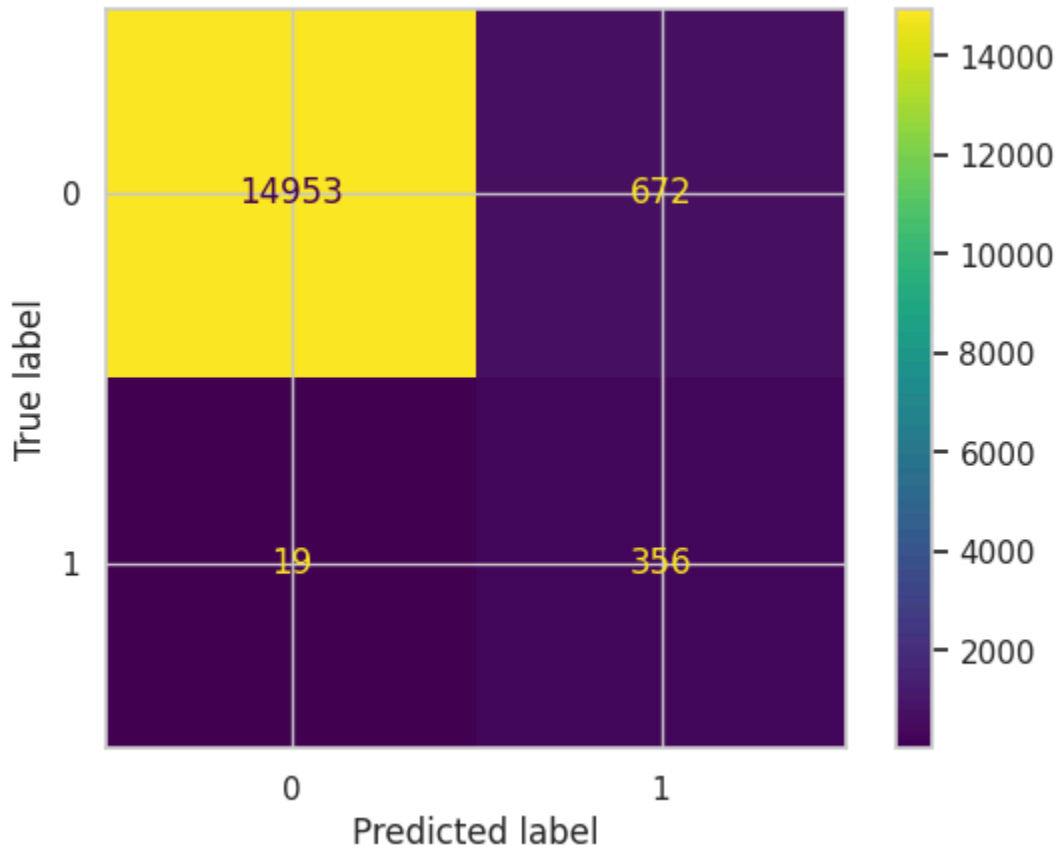


Fig. 10: best SGD - real

Figure 10, shows the SGD model, trained on the real test data set provided with the original data, correctly identified 356 APS failures while missing 19 cases (false negatives). This corresponds to a recall of approximately 93%. The model also produced 672 false positives, indicating some tendency to over-predict failures.

Figure 11 illustrates the best Logistic Regression model on the real test set.

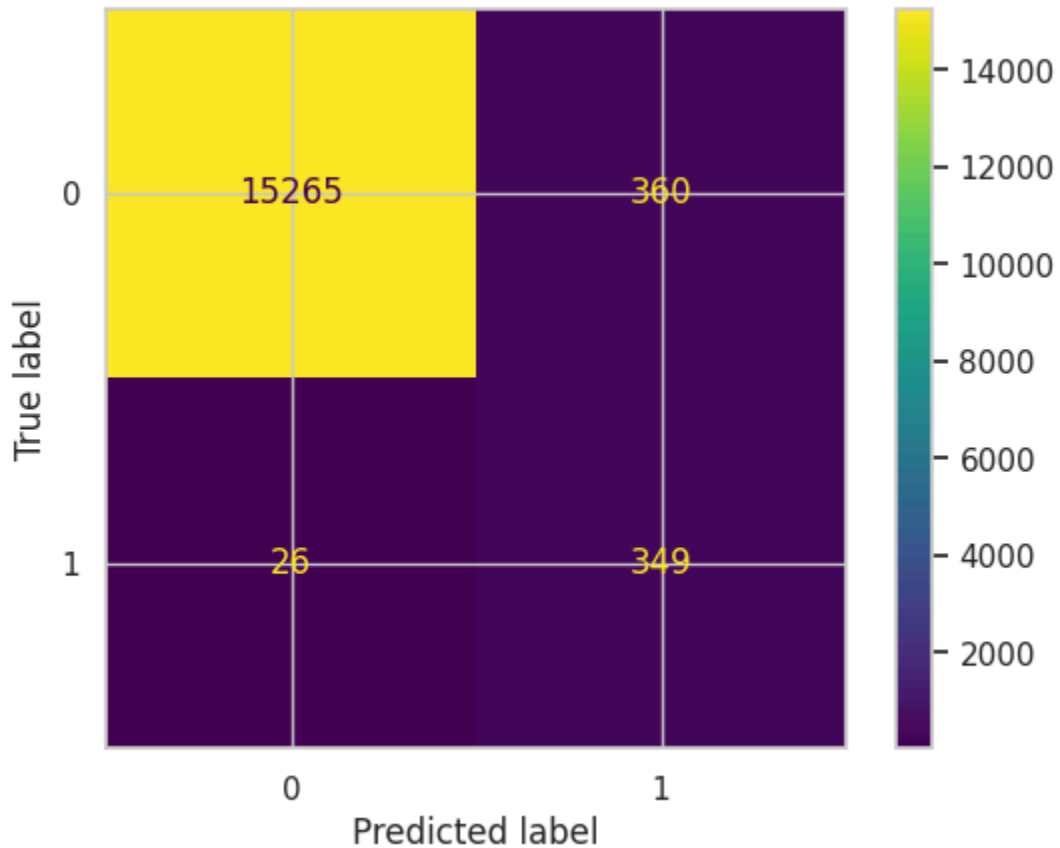


Fig. 11: best Logistic Regression - real

Finally, the Logistic Regression model shown in Figure 11, trained on the real test data set, correctly identified 349 APS failures while missing 26 (false negatives). Although minimizing false negatives is the primary objective, it is important to take into account that the number of missed failures is fairly small compared to the total number of positive cases.

The model also produced 360 false positives, nearly half the number of false positives produced by the SGD model.

4.4 Discussion of Results

Logistic Regression was selected as the final model due to its superior balance between recall and precision. Although the SGD Classifier achieved slightly higher recall, the improvement was minimal (13 vs 19 false negatives) and came at the cost of significantly lower precision and F1-score and nearly double the false positives.

The results of the confusion matrices suggest that Logistic Regression is better suited for this problem, because it effectively detects APS failures while limiting the number of false alarms. This balance is important for practical applications, because excessive false positives could lead to unnecessary inspections or maintenance actions.

Hyperparameter tuning resulted in only modest improvements in recall for both models, this indicates that the initial model selection played a larger role in performance than my tuning did.

Dimensionality reduction using PCA had little impact on model performance, this suggests that the original feature set had captured the necessary information for classification.

A limitation of this approach is the reliance on median imputation for the missing data, which might have oversimplified complex relationships in the dataset. Additionally, the severe class imbalance remains a challenge, and while class weighting helped mitigate this issue, I would explore alternative approaches such as SMOTE in future work.

One result that surprised me was that more complex models such as Random Forest and SVM did not outperform simpler linear models.

Finally, while the models performed well in terms of recall, there is still room for improvement. The presence of false negatives indicates to me that some failure patterns are still not fully captured by the models.

4.5 Practical Implications of Model Performance

Beyond evaluation metrics, it is important to consider how these models would behave in a real-world setting. In the context of truck maintenance, predictions made by the model would likely be used to trigger inspections or preventative maintenance.

A model with high recall ensures that most failures are detected, which is critical for safety and reliability. However, a high number of false positives could lead to unnecessary inspections, increasing operational costs and downtime.

The SGD Classifier, has a slightly higher recall, but would result in a larger number of unnecessary inspections due to its higher false positive rate. In contrast, the Logistic Regression model reduces this issue with higher precision while still maintaining a high level of failure detection. This, then, would make it more practical for deployment in real-world systems where resources such as time and labor are limited.

Another potentially important consideration is model interpretability. Logistic Regression provides coefficients that can be analyzed to understand how different features contribute to predictions. This can be useful for engineers and maintenance teams who want insight into which factors are associated with APS failures. While an SGD Classifier is less interpretable, which may limit their usefulness in decision-making contexts.

Overall, while both models perform well from a purely predictive standpoint, Logistic Regression offers a better balance between performance, cost, and usability.

5. Conclusion:

In this project I attempted to address the problem of predicting APS system failures in trucks using sensor data from the Scania APS Failure dataset. The highly imbalanced nature of the dataset made this challenging. Also, the high cost associated with missed failures, required that the primary objective be to minimize false negatives.

I evaluated multiple machine learning models in order to find an effective prediction model, including Logistic Regression, K-Nearest Neighbors, Support Vector Machines, Decision Trees, Random Forests, and SGD Classifier. Among these, Logistic Regression and SGD Classifier achieved the highest recall and were selected for hypertuning.

The final results demonstrated that Logistic Regression provided the best overall performance. While SGD Classifier achieved slightly higher recall, the improvement was minimal and came at the cost of significantly lower precision and while precision was not the primary focus it was important for preventing false positives. Logistic Regression maintained a high recall (approximately 93%) while achieving better precision and the highest F1-score, making it the most balanced and reliable model.

These findings resolve the project's objective, showing that it is possible to effectively detect APS failures while limiting false positives. The results suggest that linear models are well-suited for this high-dimensional classification problem and that dimensionality reduction through PCA was not necessary.

For future work, I could explore several improvements. First, techniques for better handling class imbalance, such as SMOTE or other methods, could be applied to further improve recall without significantly reducing precision. Second, I could investigate additional feature engineering approaches or feature selection methods to reduce noise in the data and improve model performance. Finally, I could explore more complex models, such as gradient boosting methods, and evaluate them to determine whether they provide further performance gains.

References:

APS Failure Dataset (Scania Trucks), 2016 Industrial Data Analysis Challenge.
UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/414/ida2016challenge>